

is *gsub(r, t, s)*. As the *man* page states, it substitutes *t* for occurrences of the regular expression *r* in the string *s*. If *s* is not given, *\$0* is used. Awk also provides a *sub()*, which is the same as *gsub*, except that it only replaces the first occurrence. For example, to replace every occurrence of one or more spaces in the file with a single tab, you could use the following code:

```
awk '{gsub(/ +/,"\\t"); print $0}' <file_name>
```

Besides these, there are several other string-manipulation functions like *index*, *length*, *match*, *strtonum*, *substr*, *toupper*, *tolower*, etc.

Time conversion

Now, let's have a look at the time conversion functions available in Awk, which provides three very useful functions to fetch or manipulate time: *system()*, *strftime()* and *mktime()*.

system(): Returns the current time-stamp, for example, `awk 'BEGIN{print system()}'`.

strftime(): Returns the time in the specified format (similar to what is used for the date command). For example, `%d-%m-%Y %H:%M:%S` can be used to return time in the "DD-MM-YYYY HH24:MI:SS" format.

mktime(): It takes a datespec of the form YYYY MM DD HH MM SS[DST] and returns a time-stamp of the same form as returned by *system()*.

The following code snippet can be used to convert the time-stamp provided in Squid logs to the standard date format:

```
awk '{s1=strftime("%d-%m-%Y %H:%M:%S",$1); print $0}' /var/log/squid/access.log
```

Often, systems administrators need to figure out a date which is a few days prior to or after the current date, which is helpful for archiving or purging log files older than a particular date. Some simple logic, along with the *strftime* and *system* functions, can make this task very trivial. Since Linux stores time as the number of seconds since epoch (midnight, January 1, 1970 GMT), and there are 86400 seconds in a day, we can easily calculate the date a few days prior to or after the current date, using the code snippets below, which return the dates 5 days after and 8 days prior to the current date:

```
awk -v days="5" 'BEGIN{print strftime("%Y-%m-%d",system()+ (days*86400))}'
```

```
awk -v days="-8" 'BEGIN{print strftime("%Y-%m-%d",system()+ (days*86400))}'
```

Arrays in Awk

Awk's support of associative arrays, combined with its ability

to treat individual columns as fields, provides a very powerful mechanism for analysing and processing data. This allows for running SQL-like grouping functions on columns of text. Before having a look at practical examples, let's briefly discuss arrays in Awk. Just like other variables in Awk, we don't have to explicitly define arrays or the type of data they would contain. An array index can be any integer or a string, or even both for a single array. Thus, an array variable could contain both the number '10' and the string 'error' as its indexes. Awk doesn't support multidimensional arrays, but we will shortly see how we can combine different fields to emulate that functionality.

Example uses of arrays

To get a count of the connections opened by a client to access the SSH service on your server, you'd use the *netstat -antp* command, whose output contains the following seven fields:

Proto	Recv-Q	Send-Q	Local Address	Foreign Address
State	PID/Program name			
tcp	0	0	::ffff:192.168.56.1:22	
	::ffff:192.168.56.1:53063	ESTABLISHED	2655/2	
tcp	0	0	::ffff:192.168.56.1:22	
	::ffff:192.168.56.1:53064	ESTABLISHED	2696/4	

To process this, let's use:

```
netstat -antp | awk -v IGNORECASE=2 -v prt=22 '{
  split($4,laddr,"."); split($5,raddr,".");
  if(laddr[5]==prt)sum[faddr[4]]+=1
}END{for(ip in sum) print ip"\t"sum[ip]}'
192.168.56.101 1
192.168.56.1 2
```

In this case, split the 4th field (destination-address) and the 5th (source-address), using the "." as a separator; take the 5th (destination port) and the 4th (source IP) indexes respectively from the resultant arrays, and generate a sum on a per-source-IP basis. We have not considered the state of the connection—whether it's OPEN, ESTABLISHED or TIME_WAIT. However, if the need is to consider the connection state, too, then we can emulate multidimensional array functionality by combining multiple fields to form an array index.

For example, by changing the port to 80, and adding the 6th field to the array index at the time of the sum calculation, we can get the source IP and connection-state-wise sum for the connections established to the Web server:

```
netstat -antp | awk -v IGNORECASE=2 -v prt=22 '{
  split($4,laddr,"."); split($5,raddr,".");
  if(laddr[5]==prt)sum[faddr[4]":"$6]+=1
}END{for(ip in sum) print ip"\t"sum[ip]}'
192.168.56.1:TIME_WAIT 1
```